



**BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH
HONOURS**

CSE3433 - SOFTWARE ARCHITECTURE

SEMESTER II: SESSION 2025/2026

Project Title:

MICROSERVICES-BASED DISASTER RELIEF COORDINATION PLATFORM

GROUP 16

COURSE LECTURER:

DR. MOHAMAD NOR HASSAN

NAME	MATRIC NUMBER
ADLY AZAMIN BIN AZMAN	S76094
MUHAMMAD UQAIL RAFIQIE BIN MOHD FARIS	S76227
AFIQ FAHIM BIN MOHD RIDZUAN	S76096

Table of Contents

Selected Architecture Style.....	3
Problem Description.....	4
System Overview.....	5
System Objectives.....	6
Functional Requirements.....	7
Non-Functional Requirements.....	8
Team Member Roles.....	9
Architecture Style Explanation.....	10
Service or Component Identification.....	11
Architecture Diagrams.....	12
I. System Context Diagram.....	12
II. Component Architecture Diagram.....	13
III. Interaction Diagram.....	14
IV. Deployment Diagram.....	15

Selected Architecture Style

A microservice design was chosen because it makes the software flexible and strong. Instead of building one giant application where everything is tangled together, the system is broken down into small independent pieces. Each piece focuses on doing just one specific job. This means that a single part can be changed, tested and launched on its own without breaking the rest of the software. If one piece stops working, the other parts keep running smoothly so users are not interrupted.

This setup also changes how information is stored. Instead of forcing the whole system to use one massive database that can easily get slow, each small piece gets its own separate database. This lets developers pick the best tools for each specific job because these pieces run separately in the cloud making it very easy to handle heavy traffic. If lots of people suddenly use one specific feature, more computer power can be added just to that one part. This keeps the software running fast and saves money because there is no need to pay for an upgrade to the entire system at once.

Problem Description

During emergencies like floods, earthquakes or disease outbreaks, it is very important for different groups to work well together. However, this is often hard to do. Many current disaster systems use older single block designs. These older designs are not flexible enough to handle the sudden rush of users and information that happens during a crisis.

A big problem with these older systems is that they can slow down or crash completely when too many people use them at once. If many victims try to report problems or ask for help at the exact same time, the system can freeze. This can cause dangerous delays for rescue teams trying to save lives.

Another issue is poor communication between groups like the government, charities, and volunteers. When information is not shared instantly, rescue workers might end up doing the same job twice or they might completely miss areas that badly need help.

Also, updating these traditional systems is very hard because all the parts are tightly linked together. Changing just one small piece can break the whole system. This makes fixing or improving the software slow and risky during an emergency.

Because of these problems, a better, more flexible system is needed. A modern design must be able to handle sudden spikes in users, share information instantly and allow different parts of the system to work and update on their own without breaking the rest of the platform.

System Overview

The new Disaster Relief Platform is a modern system built to replace older, single-block apps used in crisis management. In the past, older disaster systems had all their software parts tightly linked together, meaning a change to one small piece could break the whole system. This made updates very slow and risky during a real emergency. A major problem with these older designs is that they often slow down or crash completely when too many victims try to report problems at the exact same time.

To fix these problems, the new platform uses a microservices design that separates the main jobs into completely independent pieces. There are separate parts for managing user accounts, handling emergency reports, tracking logistics, and coordinating volunteers. Because each part focuses on doing only one specific job, a single piece can be updated or fixed on its own without breaking the rest of the software. To connect everything, a central API Gateway acts as the main front door for the app, securely taking requests from users and instantly routing them to the correct background service.

Furthermore, instead of forcing the whole system to share one massive database that can easily get slow, each small piece is given its own separate database. This setup stops data traffic jams and is incredibly useful during unpredictable disasters. If a flood happens and thousands of people send emergency reports at once, the system can automatically add more computer power just to the reporting piece without needing to upgrade the entire platform. This targeted scaling keeps the rescue tools working fast, shares information instantly, and prevents dangerous delays to save lives.

System Objectives

I. Real-time processing

The system must ensure that critical data, such as emergency reports and location updates, is shared instantly among government bodies, charities, and volunteers. This allows the system to process information immediately so rescue teams can act fast without having to wait.

II. Scalability

The system should be able to handle sudden spikes in users and information, especially during a large-scale crisis. This is achieved by making sure each small, independent part of the software can grow on its own when it needs to handle heavy traffic.

III. Loose Coupling

The software should be designed so that each part can work by itself and is not too dependent on other parts. This makes it easier to update or fix one piece of the system, such as supply tracking, without breaking the rest of the platform.

IV. Automation of coordination tasks

The platform should automate tasks like tracking the distribution of food and medicine, and assigning volunteers to specific rescue jobs. This reduces mistakes, makes the rescue process more efficient, and allows rescue workers to focus on the important job of saving lives.

V. Improved user experience

The system must be reliable and easy to use for victims, volunteers, and agencies. Keeping users connected and ensuring the platform stays online during an emergency makes the whole disaster relief process smoother, more professional, and much more effective.

Functional Requirements

I. User Access & Security

The system shall allow victims, volunteers, and agencies to create and manage their own accounts and shall use a single entry point (API Gateway) to keep all services secure.

II. Emergency Reporting

The system shall allow users to send real-time reports with their location and the type of emergency and shall allow the Reporting Service to scale independently to handle traffic spikes

III. Logistics & Supplies

The system shall provide a specific service to track and manage the distribution of food and medicine and use a separate database for supply tracking to keep the data organized and fast.

IV. Volunteer Coordination

The system shall allow volunteers to sign up and be assigned to specific rescue tasks and shall allow the volunteer service to be updated without stopping the rest of the platform.

V. Communication

The system shall share information instantly between the government, charities, and volunteers to avoid missing areas in need and shall allow different parts of the software to talk to each other to keep information up to date.

Non-Functional Requirements

I. Performance

The platform must be fast. When people send emergency reports or update their locations, there should be no waiting. API response times must be very quick so rescue operations are not delayed.

II. Reliability

The system must stay online almost 100% of the time. Because lives are at stake, the platform must be safe from totally breaking; if the volunteer part fails, the emergency reporting part must keep working normally.

III. Scalability

The system must handle huge crowds of users at once. During a sudden disaster, the emergency reporting part must be able to grow to handle thousands of requests without slowing down the rest of the app.

IV. Security

Personal details of victims and volunteers must be protected with encryption. The main front door (API Gateway) must check passwords carefully so only approved groups can see sensitive rescue data.

V. Maintainability

Programmers must be able to fix bugs or add new features to one part of the app without turning the whole system off.

Team Member Roles

Name	Role	Description
Adly Azamin bin Azman	Project Manager	Define project scope and timeline, allocate responsibilities to team members, track progress and ensure the project is delivered on schedule.
Muhammad Uqail Rafiqie bin Mohd Faris	Database Admin	Responsible for developing, managing, and maintaining the database system. Controls user permissions, protects data security and handles backup and recovery processes.
Afiq Fahim bin Mohd Ridzuan	System Architect	Determine how services are separated, plan communication between services, select technologies and frameworks, ensure scalability and stability.

Architecture Style Explanation

The system uses a Microservices Architecture. This means the app is built as a set of small, separate pieces that do not rely heavily on each other.

I. Loose Coupling

The pieces work independently. The Volunteer part does not need to know how the Logistics part is built. They only talk to each other using simple, agreed-upon rules.

II. Independent Deployability

Developers can update or fix a specific piece without having to restart the entire platform.

III. Decentralized Data Management

Instead of sharing one giant database, each piece manages its own database. This stops data traffic jams and lets developers use the best type of database for each specific job.

IV. Fault Isolation

If a bug causes the communication piece to crash, the problem is trapped. The Emergency Reporting and Logistics pieces will continue to work normally so critical rescue operations do not stop.

V. Granular Scalability

Computer power can be added exactly where it is needed. If thousands of people are reporting a flood, only the Reporting piece is made bigger, which saves money and resources.

Service or Component Identification

I. API Gateway

The main front door for all mobile and web users. It checks security and directs traffic to the correct internal service.

II. User Management Service

Handles sign-ups, logins, and controls what victims, volunteers, and agencies are allowed to do.

III. Emergency Reporting Service

Collects crisis reports, saving important data like the user's location, how bad the emergency is, and what type it is.

IV. Logistics & Supply Service

Tracks food, water, and medicine to see where they are currently located and where they need to go.

V. Volunteer Coordination Service

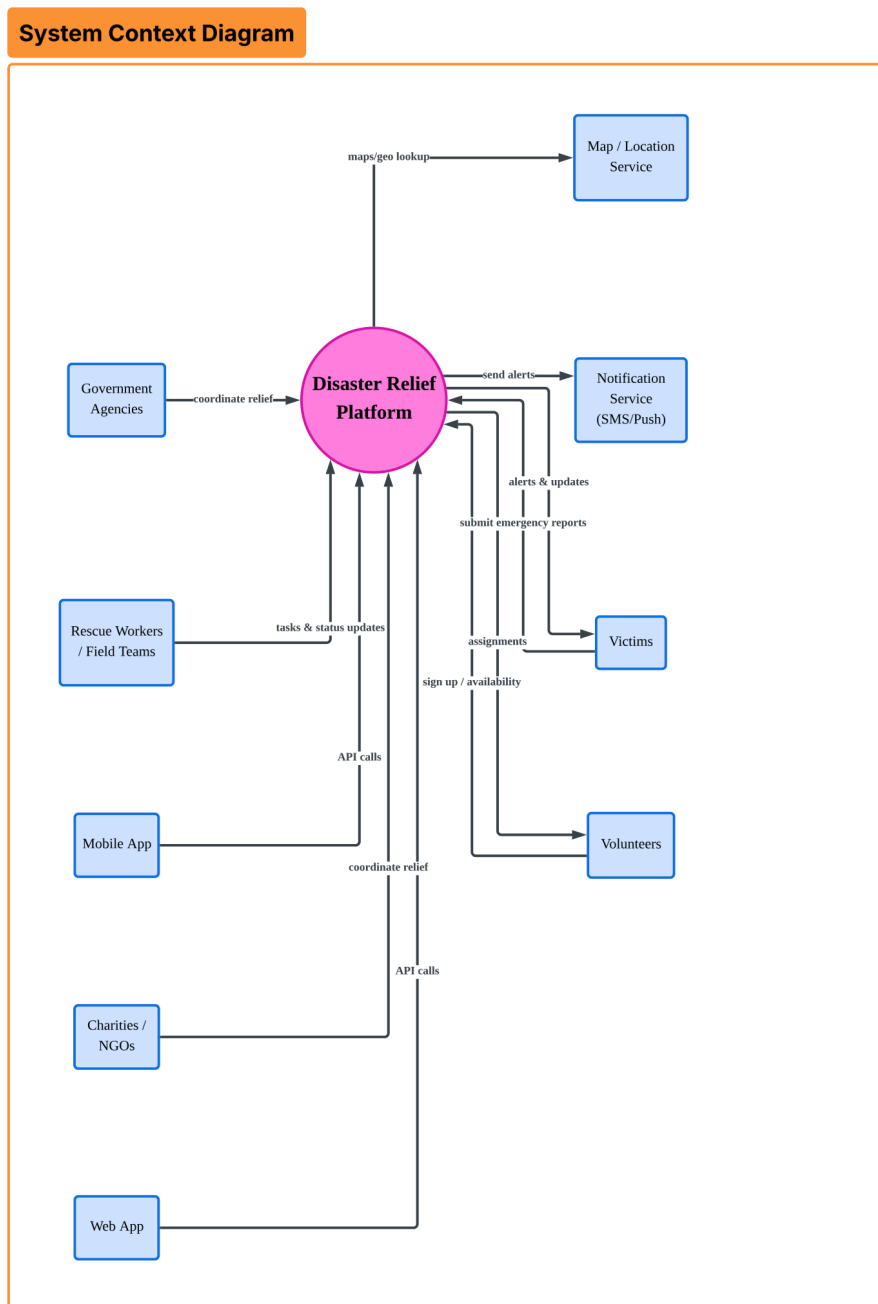
Keeps track of volunteers, their skills, and matches them to jobs where they are needed.

VI. Communication Service

Sends real-time alerts and messages between the government, charities, and rescue workers in the field.

Architecture Diagrams

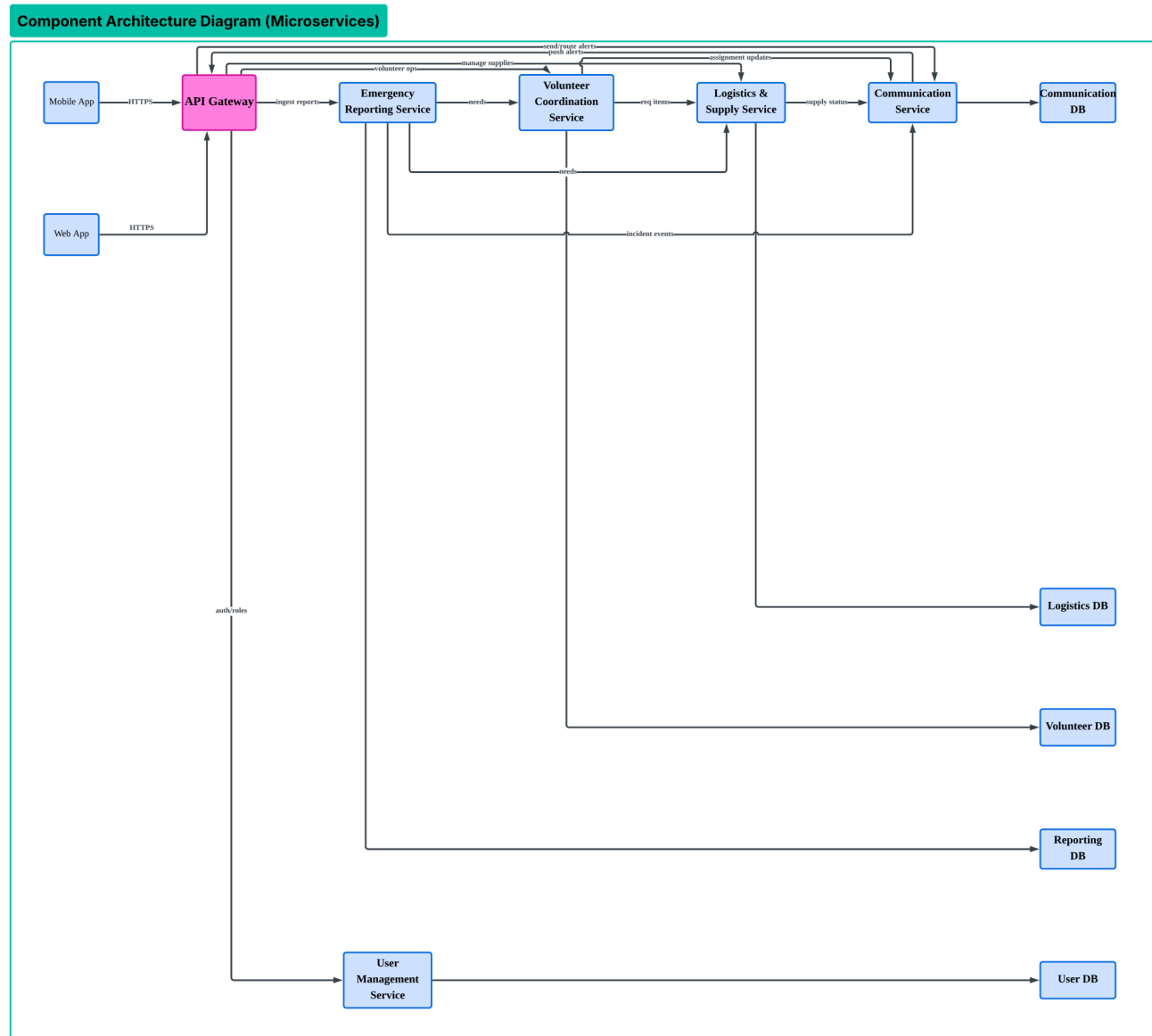
I. System Context Diagram



The above figure shows the high-level context of the Disaster Relief Platform, where victims, volunteers and agencies interact with the system to report emergencies, manage resources and coordinate tasks.

The platform also connects with external systems such as the Map System for location data and the SMS System for notifications. These interactions support real-time communication, fast response and efficient disaster management.

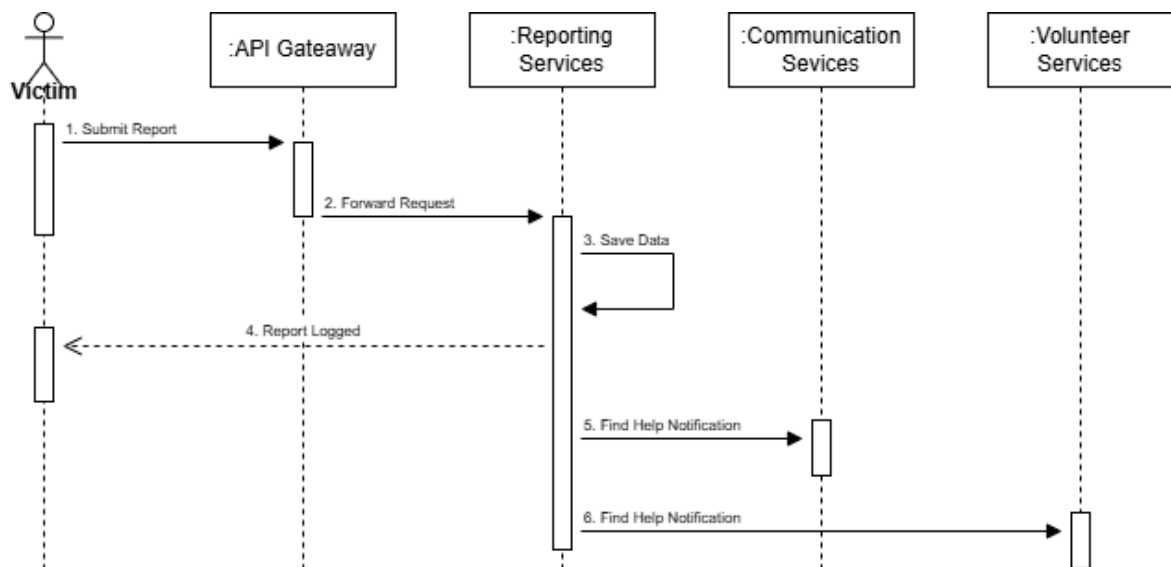
II. Component Architecture Diagram



The above figure illustrates the component architecture of the Disaster Relief Platform using a microservices design. The API Gateway acts as a central entry point that routes requests to services such as User, Reporting, Logistics, Volunteer and Communication services.

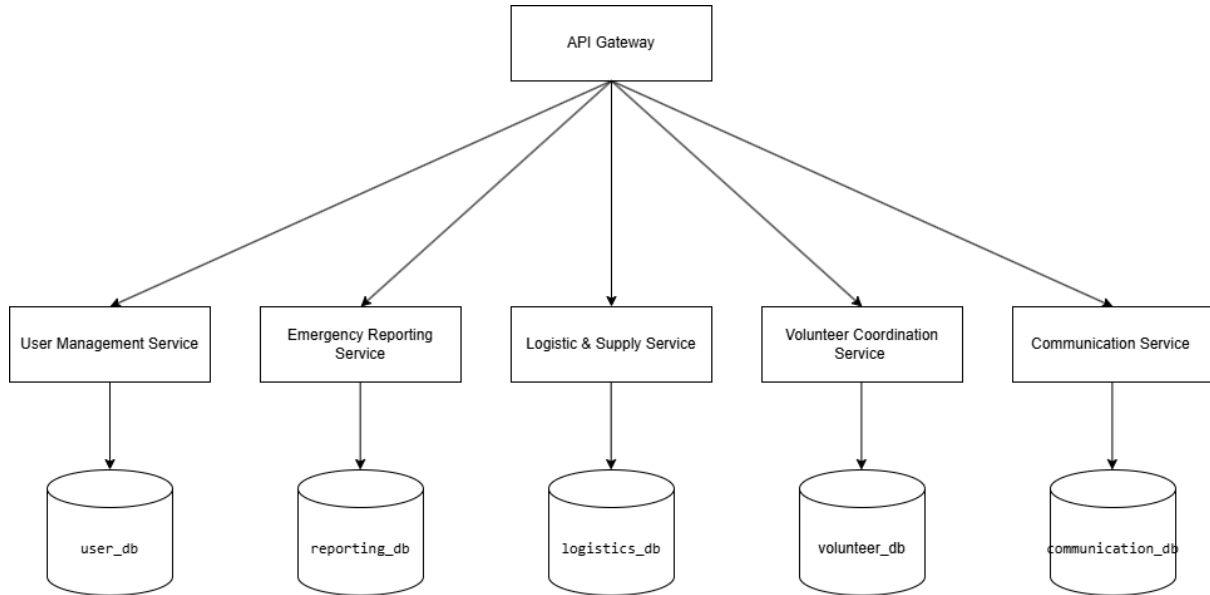
Each service has its own database, ensuring better scalability, reliability and independent operation of system components.

III. Interaction Diagram



The **Interaction Diagram** shows the step-by-step process of how the platform handles an emergency. It starts when a **Victim** sends a report through the **API Gateway**, which acts as the system's secure entry point. This gateway forwards the request to the **Reporting Service**, where the incident details are saved to a private database. After the data is stored, the system sends a "Report Logged" confirmation back to the **Victim** to let them know help is on the way. To finish the process, the **Reporting Service** automatically alerts the **Communication** and **Volunteer Services** to start organizing rescue efforts and finding available help instantly.

IV. Deployment Diagram



The **Deployment Diagram** shows how the platform is structured on a server to stay fast and reliable. At the top, the **API Gateway** acts as the main entry point, securely directing all user traffic to the right place. Below that, the system is split into five independent services(**User**, **Reporting**, **Logistics**, **Volunteer**, and **Communication**), which allows the platform to keep running even if one part has a problem (microservices). To prevent data slowdowns, each service has its own dedicated **private database**, ensuring that information stays organized and secure during a major disaster.